
```python
import sympy as sp
from kinematics.ik_solver import generate_ik

# Define robot parameters and target
l1, l2, l3 = sp.symbols('l1 l2 l3')
target_pose = sp.Matrix([[0.8, 0.2, 0.1], [0, 1, 0], [0.6, 0, 1]])

# Generate closed-form IK solver
# AI assisted derivation...
ik_solution = generate_ik(target_pose, [l1, l2, l3], method='analytical')
print(ik_solution.theta)
# Output: (theta1: [0.15, 0.22], theta2: [1.2, 1.8], ...)
```

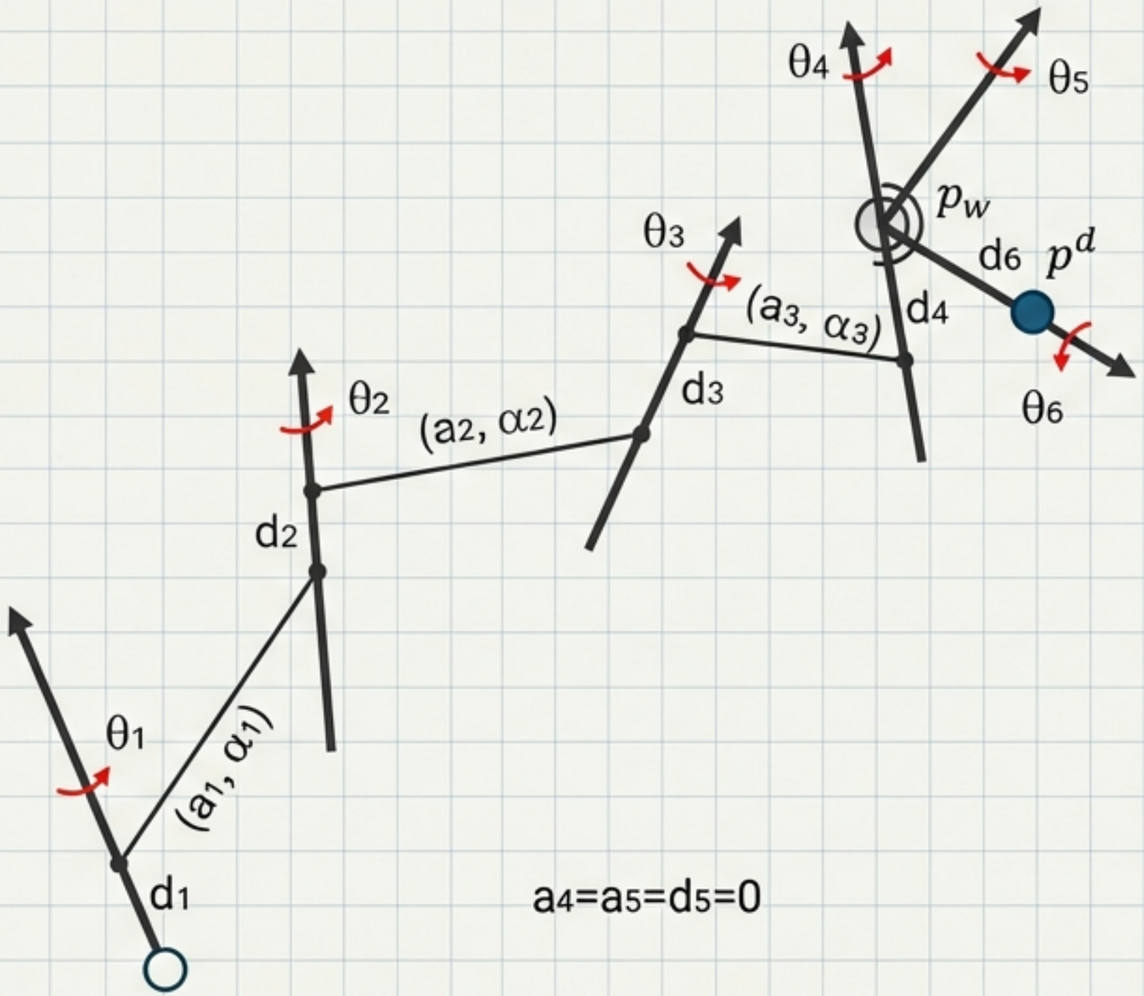
Large Language Model Assisted Human-AI Collaborative Development of Analytical Inverse Kinematics Solvers

Deriving Robust, Closed-Form Solvers in Minutes, Not Weeks.

Hai-Jun Su | Department of Mechanical and Aerospace Engineering, The Ohio State University

The Inverse Kinematics Bottleneck

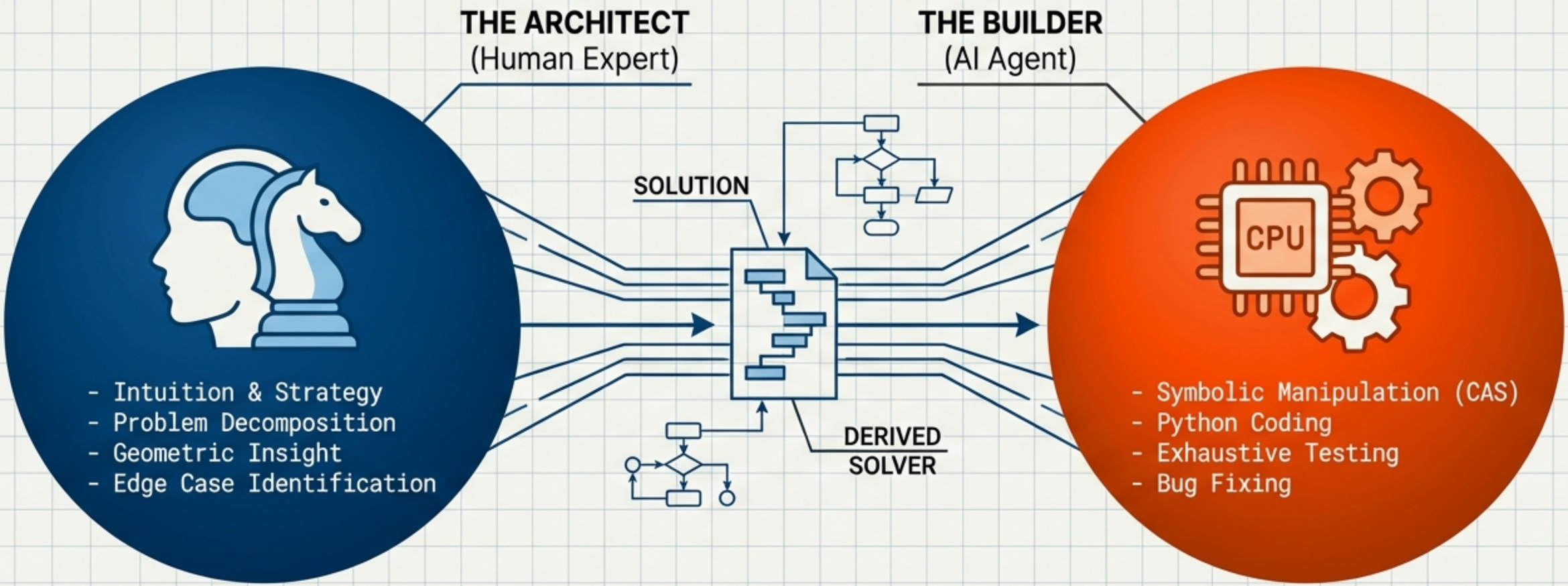
The trade-off between completeness, speed, and development effort.



Method	Performance	Pain Point
Numerical (Jacobian)	Flexible	Single solution only. Fails at singularities.
Homotopy Continuation	Complete (Finds all roots)	Too slow (Seconds vs ms). Not real-time.
Analytic/Symbolic	Fast & Complete (The Gold Standard)	Requires weeks of expert manual derivation.
Learning (IKFlow)	Fast	Approximate only. No guarantees.

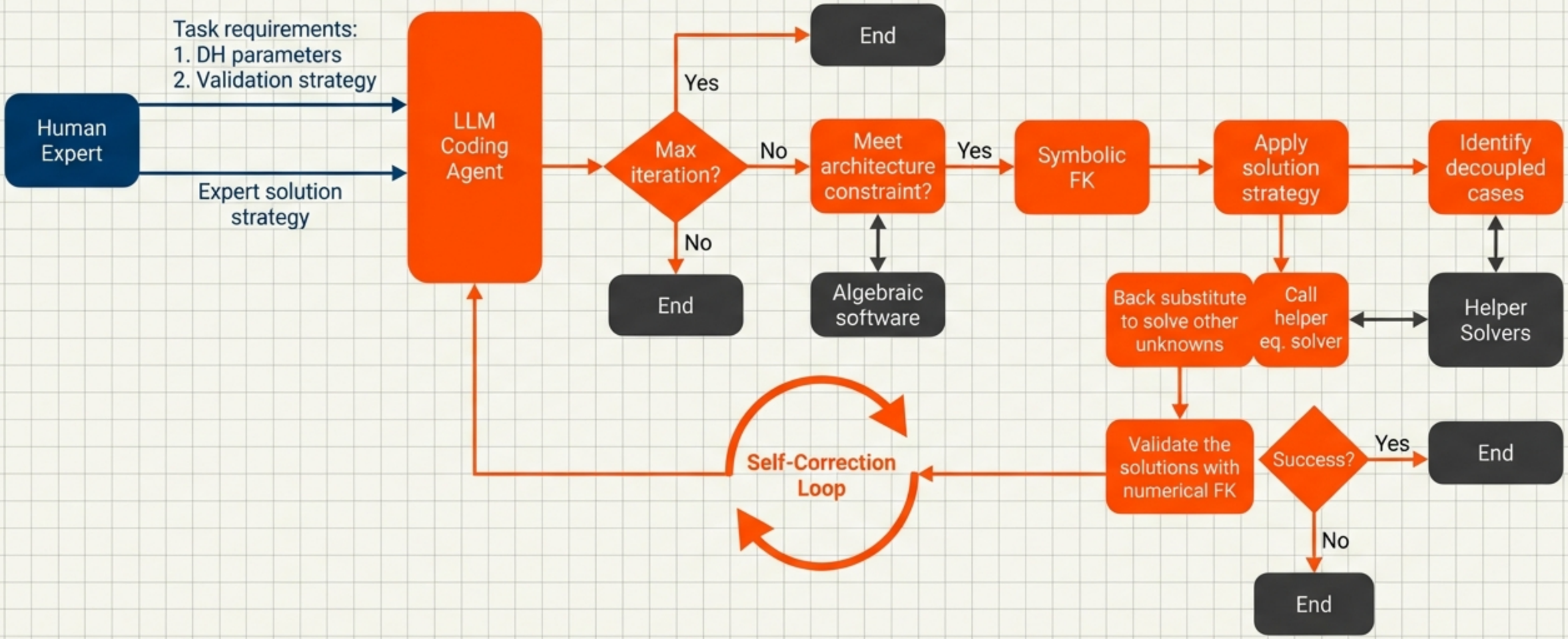
A New Division of Labor: Architect vs. Builder

The shift from manual derivation to high-level strategy and automated execution.



Shift: Manual Coding --> Supervisory Prompting

The Expert-Guided Generative Workflow



The Foundation: Verified Helper Functions

Pre-validated mathematical building blocks prevent hallucination.

solve_trig_eq

Description: Solves a*cos(x) + b*sin(x) + c = 0

Method: Weierstrass substitution

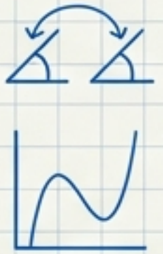
Stability: Handles collapsed constants.

solve_trig_sys

Description: Coupled two-angle systems

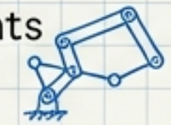
Method: Reduces to quartic polynomial

Output: Up to 4 solutions



solve_bilinear_sys

Description: Complex parallel joint constraints



Method: Sylvester resultants & Generalized Eigenvalues

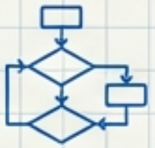
Output: Up to 8 solutions

$$\begin{bmatrix} A \\ \vdots \\ B \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

solve_trig_sys_single

Description: Linear systems for single angles

Method: Matrix inversion / degradation checks



☑ **Library Status:** Stress-tested for singularities (0=0) and numerical instability.

Encoding Expert Intuition into Prompts

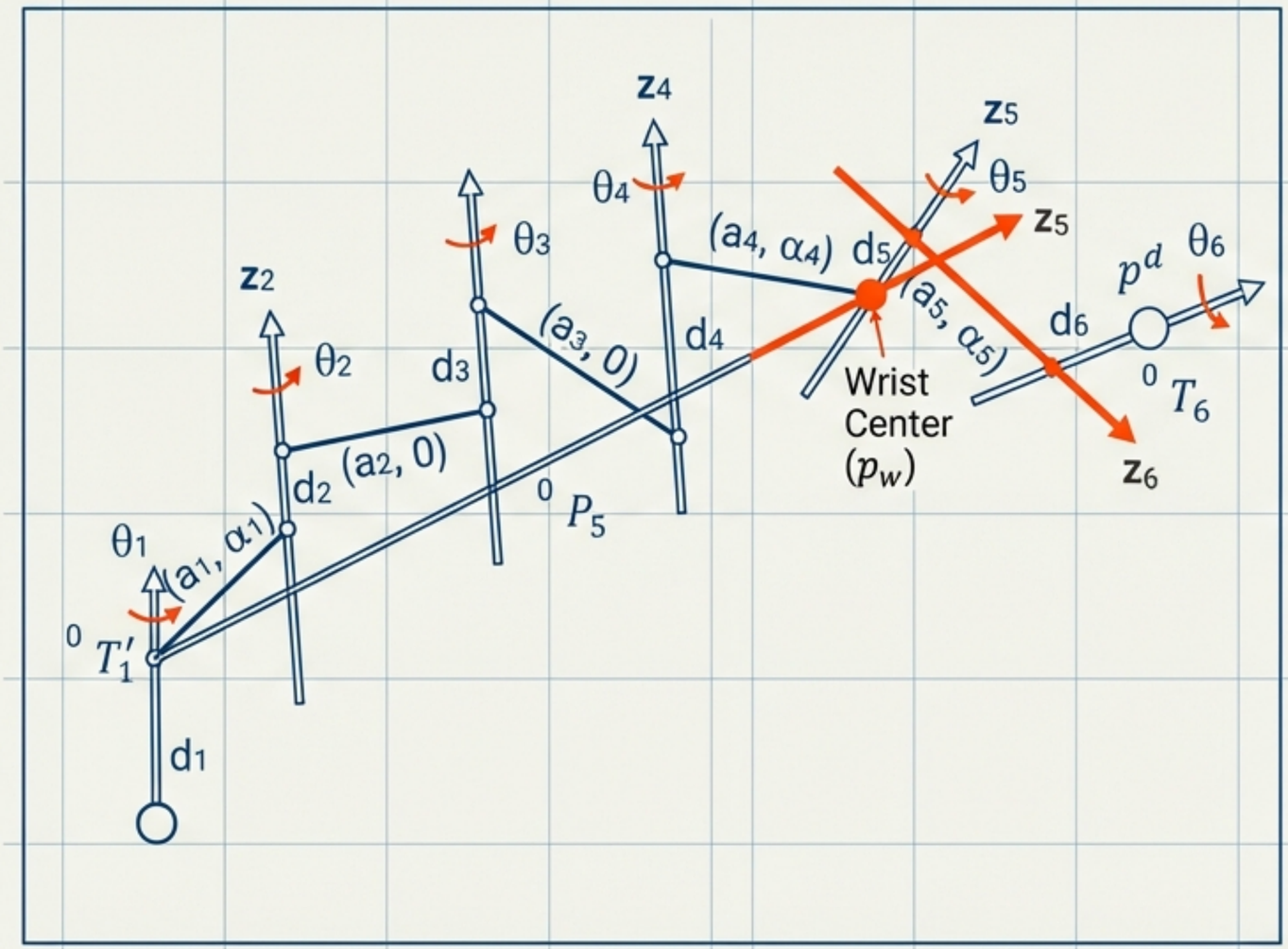
```
[CONTEXT]
You are an analytical robotics expert to develop, and premtanite
organization, public, and instructive techniical robots.

[TASK]
Write a Python script for general 6R robots with spherical
wrists, to menont the validat in enonomuicty and Jently robot,
thetas, revalnos, and, nandrovining planet...

[THE STRATEGIC HINT]
Decouple theta_1 from theta_3 by moving the transformation of
joint 1 to the LHS... Apply sin^2 + cos^2 = 1 whenever a
quadratic balloon starts forming.

[REQUIREMENTS]
Validate with 200 random samples. Handle singularities.
```

The 'Secret Sauce': The Human provides the math strategy; the AI executes the derivation.

Human-in-the-loop prompt engineering.

Case Study 1: Spherical Wrist Architecture

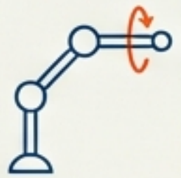
Solving the most common industrial robot configuration (>90%).



Decomposition Strategy

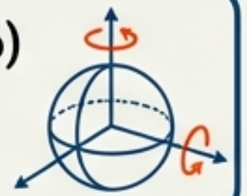
Step 1: Position (Joints 1-3)

- Use Wrist Center (p_w)
- Reduces to Linear Form: A*cos(theta) + B*sin(theta) = C



Step 2: Orientation (Joints 4-6)

- Solve Theta_5 first (via r33)
- Resolve Theta_4 / Theta_6

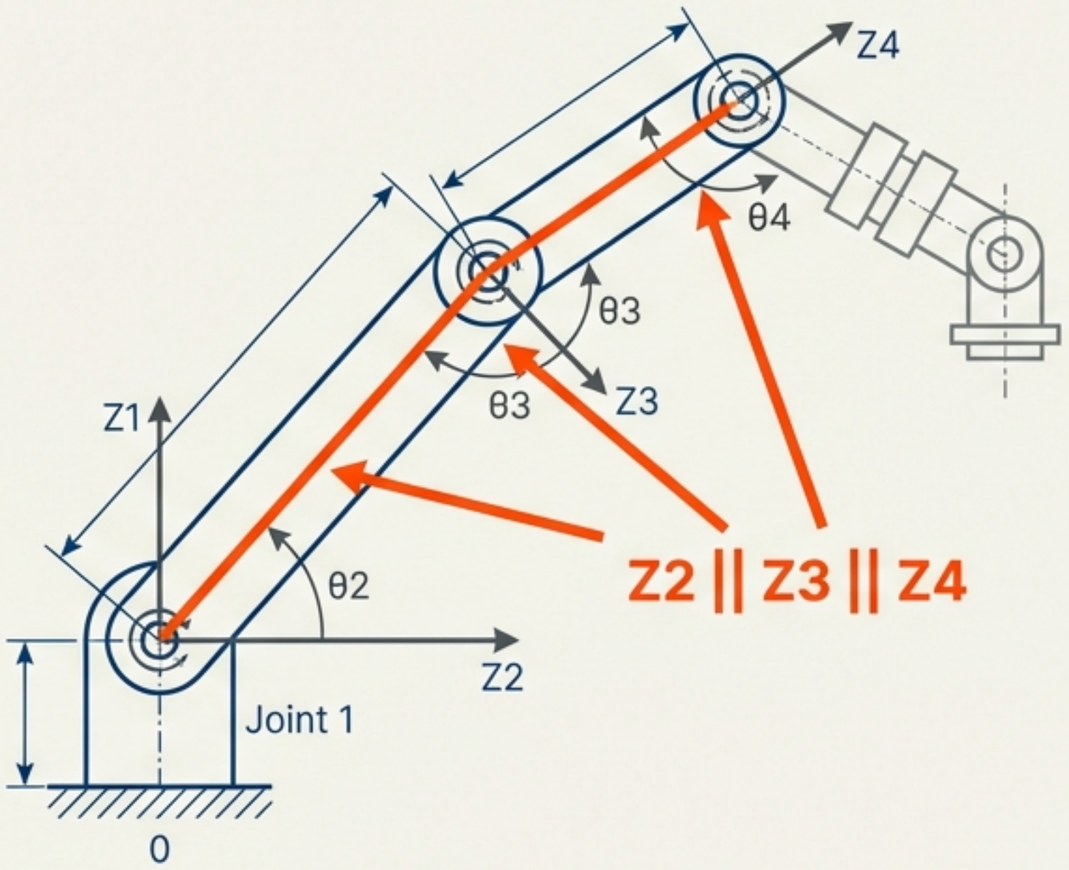


Result Box

Fully Analytic. Up to 8 Solutions.

Case Study 2: Parallel Joint Structures

Solving for collaborative robots (Cobots).



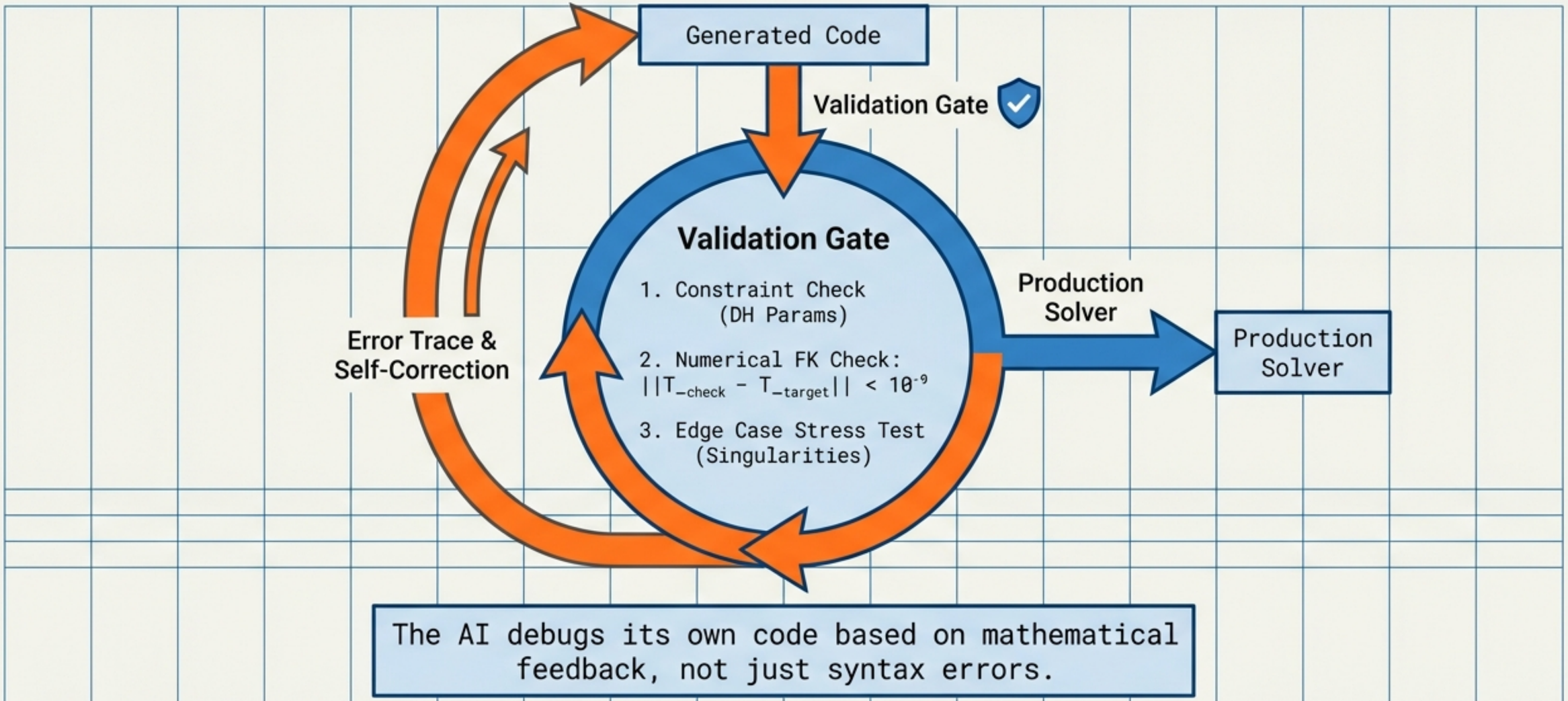
Strategy Breakdown

1. **Bilinear System Check**
   - Solve Joints 1 & 6 simultaneously.
   - Constraint: Projections on parallel plane.
   - Tool: `solve_bilinear_sys` (Resultant Theory)
2. **Orientation Fix**
   - Solve Joint 5.
3. **Planar 3R Reduction**
   - Solve Joints 2, 3, 4 as a 2D planar problem.

"AI correctly identifies and applies Sylvester Resultants for variable elimination." [Graphite] Roboto Mono.

The Iterative Validation Loop

Ensuring industrial-grade reliability.



Benchmarks: Spherical Wrist

Tested on 723 Real Industrial Robots + 1000 Random Configurations.

100%

Success Rate

Across all valid architectures.

1.2 ms

Average Solve Time

Python Implementation.

10^-9

Max Position Error

Near Machine Precision.

Benchmarks: Parallel Joint Structures

Tested on 119 Collaborative Robots + 100 Random Configurations.

100%

Success Rate

Handles complex bilinear systems.

1.1 - 1.7 ms

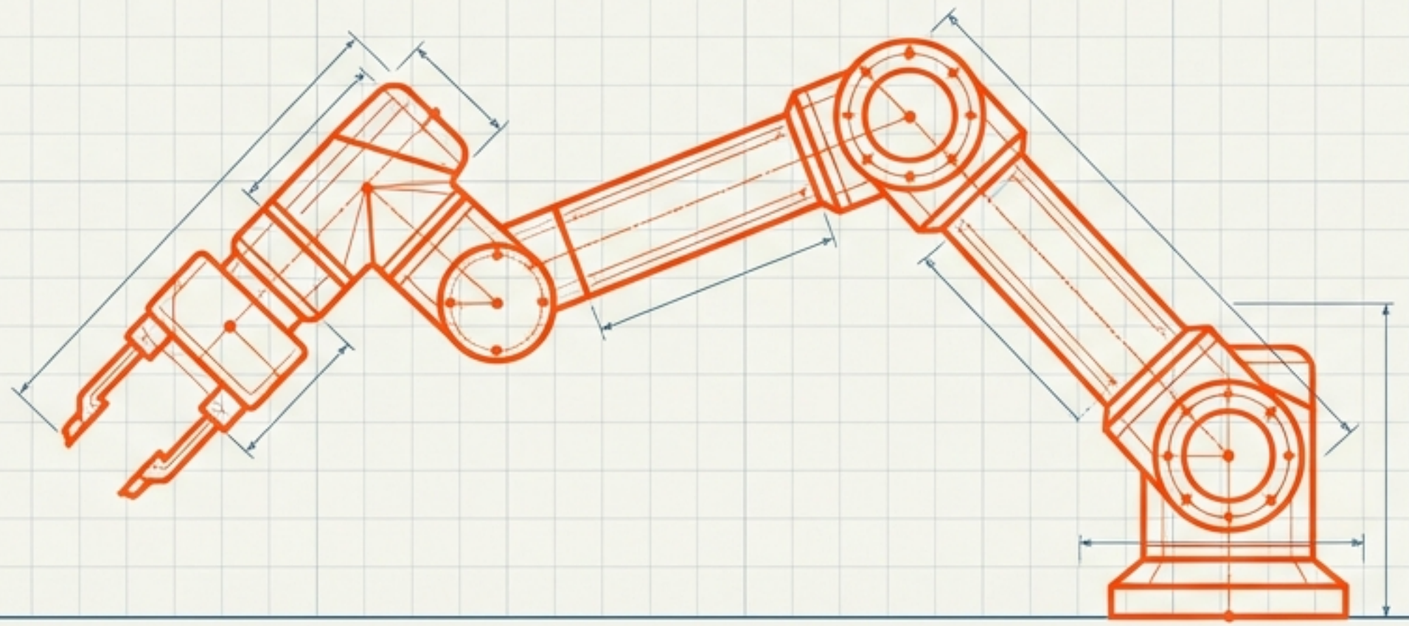
Average Solve Time

Depending on complexity.

10^-9

Max Position Error

Numerical stability maintained.



Outperforming State-of-the-Art

Method	Completeness	Speed (Runtime)	Dev Time	Accuracy
This Work (AI-Assisted)	All Solutions	~1 ms	Minutes	Exact
Jacobian (Numerical)	Single Only	Iterative	Minutes	Depends on convergence
Homotopy	All Solutions	Slow (Seconds)	Minutes	Exact
Manual Analytic	All Solutions	<1 ms	Weeks	Exact
Learning (IKFlow)	Probabilistic	<1 ms	Training Days	Approx (~10^-3)

Advantages & Limitations

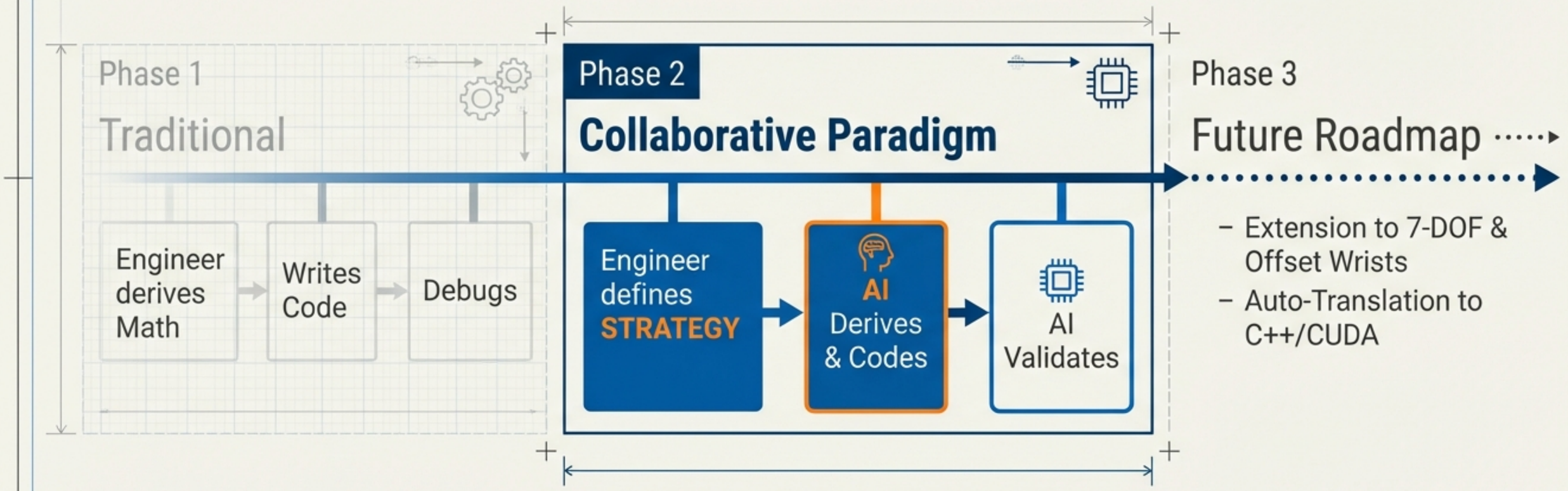
ADVANTAGES

+ Speed of Development: Minutes vs Weeks.
+ Generality: Works for arbitrary DH parameters within architecture.
+ Rigor: Mathematical exactness (Resultants, Weierstrass).
+ Zero Training Data Required.

LIMITATIONS

- Runtime Environment: Python (1-2ms). C++ port needed for <100µs loops.
- Human-in-the-Loop: Requires expert ‘Strategy’ inputs; not fully autonomous invention.

A New Blueprint for AI-Assisted Engineering



Human Intuition + AI Execution = Accelerated Innovation

1. **100% Success** on Industrial Architectures.
2. **Zero** Training Data.
3. **Minutes** of Development Time.

Get the Code & Prompts:

GitHub:
haijunsu-osu/llm_ik_6r_wrist

Source: Large Language Model Assisted Human-AI Collaborative Development of Analytical Inverse Kinematics Solvers, Hai-Jun Su (2026).